

## Task 1. Understanding NoSQL Databases

### a) Define NoSQL databases and describe their key features.

NoSQL database are non-relational database systems, designed for flexibility, scalability and performance. It stores and retrieves data in ways that differ from traditional relational databases (SQL). It supports flexible, schema-less data models such as key-value, document, column-family, or graph formats. It handles large volumes of unstructured, semi-structured or rapidly changing data. NoSQL is often used in big data and real-time web applications.

#### Features:

- 1) **Schema agnostic:** No predefined schema required, allowing flexibility as data structures evolve. It can be stored in formats like JSON, XML etc. It is easy to adapt to changes in application requirements.
- 2) **Data Model:** NoSQL uses collections and documents instead of tables and rows. It uses documents (similar to JSON objects) to store data. Each document can have a different structure. This allows storing diverse types of data within the same database. Data can be stored as
  - a. Documents
  - b. Key-Value pairs
  - c. Column family
  - d. Graph databases.
- 3) **Cost-effective:** Designed to run on low-cost commodity hardware, especially when scaling
- 4) **Non-ACID compliance:** Prioritize performance and scalability over ACID compliance. ACID stands for atomicity, Consistency, Isolation and Durability.
- 5) **Horizontal Scalability:** NoSQL is designed to scale out by adding more servers and supporting distributed architecture. Thus, it is suitable for cloud environment.
- 6) **High Performance:** NoSQL is designed for fast reads/writes with large data volumes. It supports caching and in-memory operations for low-latency access.
- 7) **Relatively new technology:** Supports dynamic and changing data, ideal for modern applications, that frequently change. It empowers full stack developers to build applications, run analytics and manage databases. (Less mature compared to SQL databases, with varying levels of support.)

### b) Compare NoSQL and SQL databases by creating a table with at least 4 parameters (e.g., Schema, Scalability, Data Storage, Use cases)

SQL is a relational database which uses tables with rows and columns. It is ideal when data integrity and complex queries are essential.

While NoSQL is a non-relational database which uses documents, key-value, graph or wide-columns as a data model. It is ideal for fast, scalable storage of unstructured, semi-structured or varied data.

Please find the comparison in the table below:

|                        | <b>NoSQL Database</b>                                                                                                                                                                                                                         | <b>SQL Database</b>                                                                                                                                                  |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Schema</b>          | NoSQL database supports dynamic flexible schema. It is ideal for unstructured or semi-structured schemas. The data model contains documents, key-value, graph or wide-column.<br>The query language varies, e.g JSON like queries for MongoDB | SQL database supports fixed, predefined schema. The data model contains Tables with rows and columns.<br>The query language used is Structured Query Language (SQL). |
| <b>Scalability</b>     | NoSQL is horizontally scalable. We can scale it out by adding more servers.                                                                                                                                                                   | SQL is vertically scalable. We can add more power to a single server.                                                                                                |
| <b>Data Storage</b>    | NoSQL database stores data such as documents, key-value pairs, wide-columns, or graphs                                                                                                                                                        | In SQL database, data is stored in tables with rows and columns.                                                                                                     |
| <b>Use cases</b>       | NoSQL database is best for unstructured or semi-structured data, big data, and real-time applications. e.g. Content management, social networks                                                                                               | SQL database is best for structured data, transactional systems. It is ideal when data integrity and complex queries are essential.<br>e.g. Banking system, ERP      |
| <b>ACID Compliance</b> | Prioritize performance and scalability over ACID compliance. Generally, supports BASE (Basically Available, Soft state, Eventually consistent)                                                                                                | Strong ACID compliance (Atomicity, Consistency, Isolation, Durability)                                                                                               |

**c) Mention two real-world use cases where NoSQL databases are more suitable than SQL databases.**

Below are the two real-world use cases for NoSQL databases.

**1) E-Commerce Platforms:**

NoSQL databases are ideal for E-Commerce platforms like Amazon, eBay. In such E-Commerce platforms, product catalogs contain items with different attributes. Products come in different types like clothes, electronic items, books etc. Each product has different attributes. SQL Database has fixed, rigid schema which is not efficient to handle this scenario. As NoSQL database is capable of handling unstructured and semi-structured data, it is effective to use NoSQL database for E-Commerce applications.

- Schema flexibility for varying product details. It allows storing products with different fields in the same collection.

- The performance is fast and low latency during high-traffic sales on E-Commerce websites
- It is easy to scale as the data grows. NoSQL scales horizontally and it can handle large users loads and heavy write/read operations during peak hours.
- No redesign of database, every time a new product is added.
- In E-Commerce, users need to search for data e.g. by catalog, price etc. NoSQL is search oriented and allows fast search and filtering (unlike SQL where complex search can be slow). NoSQL also allows indexing and querying on various fields.

## 2) Social Media Platforms:

NoSQL is an efficient database option for Social Media applications like Facebook, Twitter, Instagram etc. Social media generates large volume of unstructured or semi-structured data like posts, comments, images, videos etc. The data is huge, unstructured and it changes frequently. The data also should be delivered in real time.

- On Social Media platforms, users expect instant feedback, likes, and comments to be updated. NoSQL handles high-speed read/write operations on the data.
- It supports flexible data models like documents.
- Social platforms have millions of users and hence amount of data grows rapidly. NoSQL database distributes data across multiple servers and supports scalability.
- NoSQL's flexible schema supports varying data formats like text, images, reactions etc. New fields or data types can be added without affecting existing data.

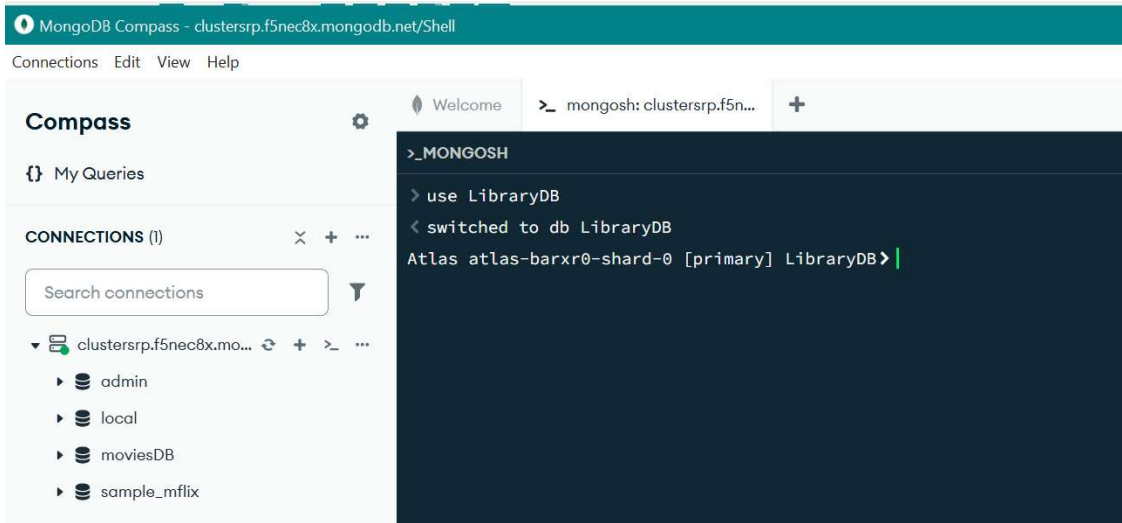
## Task 2. Working with MongoDB Collections and Documents

### a) Create a MongoDB database called LibraryDB.

The command below, is used on MongoDB Shell (mongosh) to create a new database called 'LibraryDB'

**use LibraryDB**

Please find the screenshot below.



**b) Create a collection Books and insert at least three documents with fields like title, author, year, and genre.**

The command below, is used on MongoDB Shell (mongosh) to create a new collection of 'Books' and inserting documents in that collection.

```
db.Books.insertMany([
  {
    title: "Harry Potter and the Philosopher's Stone",
    author: "J.K. Rowling",
    year: 1997,
    genre: "Fantasy Fiction",
    pages: 223,
    isbn: "978-0-7475-3269-9"
  },
  {
    title: "Harry Potter and the Chamber of Secrets",
    author: "J.K. Rowling",
    year: 1998,
    genre: "Fantasy Fiction",
    pages: 251,
    isbn: "0-7475-3849-2"
  },
  {
    title: "Harry Potter and the Prisoner of Azkaban",
    author: "J.K. Rowling",
    year: 1999,
```

```
    genre: "Fantasy Fiction",
    pages: 317,
    isbn: "0-7475-4215-5"
  },
  {
    title: "Harry Potter and the Goblet of Fire",
    author: "J.K. Rowling",
    year: 2000,
    genre: "Fantasy Fiction",
    pages: 636,
    isbn: "0-7475-5079-4"
  },
  {
    title: "Harry Potter and the Order of the Phoenix",
    author: "J.K. Rowling",
    year: 2003,
    genre: "Fantasy Fiction",
    pages: 766,
    isbn: "0-7475-5100-6"
  },
  {
    title: "Harry Potter and the Half-Blood Prince",
    author: "J.K. Rowling",
    year: 2005,
    genre: "Fantasy Fiction",
    pages: 672,
    isbn: "0-7475-8108-8"
  },
  {
    title: "Harry Potter and the Deathly Hallows",
    author: "J.K. Rowling",
    year: 2007,
    genre: "Fantasy Fiction",
    pages: 607,
    isbn: "0-7475-9105-9"
  },
  {
    title: "The Ballad of Songbirds and Snakes",
    author: "Suzanne Collins",
    year: 2020,
```

```

        genre: "Science Fiction",
        pages: 528,
        isbn: "9780702300172"
    },
    {
        title: "Sunrise on the Reaping",
        author: "Suzanne Collins",
        year: 2025,
        genre: "Science Fiction",
        pages: 400,
        isbn: "9781546171461"
    }
]

```

Please find the screenshots below.

The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' list with a search bar and a tree view showing the 'clustersrp.f5nec8x.mongod.net/Shell' connection. The main panel shows the 'My Queries' tab with a query editor. The query editor displays the following JavaScript code:

```

>_MONGOSH
> use LibraryDB
< switched to db LibraryDB
> db.Books.insertMany([
  {
    title: "Harry Potter and the Philosopher's Stone",
    author: "J.K. Rowling",
    year: 1997,
    genre: "Fantasy Fiction",
    pages: 223,
    isbn: "978-0-7475-3269-9"
  },
  {
    title: "Harry Potter and the Chamber of Secrets",
    author: "J.K. Rowling",
    year: 1998,
    genre: "Fantasy Fiction",
    pages: 251,
    isbn: "0-7475-3849-2"
  },
  {
    title: "Harry Potter and the Prisoner of Azkaban",
    author: "J.K. Rowling",
    year: 1999,
    genre: "Fantasy Fiction",
    pages: 317,
    isbn: "0-7475-4215-5"
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68e3732c10a9404ec288ceb4'),
    '1': ObjectId('68e3732c10a9404ec288ceb5'),
    '2': ObjectId('68e3732c10a9404ec288ceb6')
  }
}
Atlas atlas-barxr0-shard-0 [primary] LibraryDB>

```

## Compass

{ } My Queries

CONNECTIONS (1)

- clustersrp.f5nec8x.mo...
  - admin
  - local
  - moviesDB
  - sample\_mflix

&gt;\_MONGOSH

```
'1': ObjectId('68e3732c10a9404ec288ceb5'),
'2': ObjectId('68e3732c10a9404ec288ceb6')
}
}
> db.Books.insertMany([
  {
    title: "Harry Potter and the Goblet of Fire",
    author: "J.K. Rowling",
    year: 2000,
    genre: "Fantasy Fiction",
    pages: 636,
    isbn: "0-7475-5079-4"
  },
  {
    title: "Harry Potter and the Order of the Phoenix",
    author: "J.K. Rowling",
    year: 2003,
    genre: "Fantasy Fiction",
    pages: 766,
    isbn: "0-7475-5100-6"
  },
  {
    title: "Harry Potter and the Half-Blood Prince",
    author: "J.K. Rowling",
    year: 2005,
    genre: "Fantasy Fiction",
    pages: 672,
    isbn: "0-7475-8108-8"
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68e3737a10a9404ec288ceb7'),
    '1': ObjectId('68e3737a10a9404ec288ceb8'),
    '2': ObjectId('68e3737a10a9404ec288ceb9')
  }
}
Atlas atlas-barxr0-shard-0 [primary] LibraryDB>
```

## Compass

{ My Queries

CONNECTIONS (1)

Search connections

- ▼ clustersrp.f5nec8x.mo...
  - ▶ admin
  - ▶ local
  - ▶ moviesDB
  - ▶ sample\_mflix

&gt; \_MONGOSH

```
> db.Books.insertMany([
  {
    title: "Harry Potter and the Deathly Hallows",
    author: "J.K. Rowling",
    year: 2007,
    genre: "Fantasy Fiction",
    pages: 607,
    isbn: "0-7475-9105-9"
  },
  {
    title: "The Ballad of Songbirds and Snakes",
    author: "Suzanne Collins",
    year: 2020,
    genre: "Science Fiction",
    pages: 528,
    isbn: "9780702300172"
  },
  {
    title: "Sunrise on the Reaping",
    author: "Suzanne Collins",
    year: 2025,
    genre: "Science Fiction",
    pages: 400,
    isbn: "9781546171461"
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68e373c710a9404ec288ceba'),
    '1': ObjectId('68e373c710a9404ec288cebb'),
    '2': ObjectId('68e373c710a9404ec288cebc')
  }
}
Atlas atlas-barxr0-shard-0 [primary] LibraryDB>
```

- c) Use the find command to fetch all books where year is greater than 2000.  
Below command is used to fetch the books where year is greater than 2000.

```
db.Books.find({year:{$gt:2000}}).pretty()
```



## Compass

My Queries

CONNECTIONS (1)

Search connections

clustersrp.f5nec8x.mongodb.net

- admin
- local
- moviesDB
- sample\_mflix

&gt;\_MONGOSH

&gt; db.Books.find({year:{&gt;2000}}).pretty()

```
< [
  {
    _id: ObjectId('68e3737a10a9404ec288ceb8'),
    title: 'Harry Potter and the Order of the Phoenix',
    author: 'J.K. Rowling',
    year: 2003,
    genre: 'Fantasy Fiction',
    pages: 766,
    isbn: '0-7475-5100-6'
  },
  {
    _id: ObjectId('68e3737a10a9404ec288ceb9'),
    title: 'Harry Potter and the Half-Blood Prince',
    author: 'J.K. Rowling',
    year: 2005,
    genre: 'Fantasy Fiction',
    pages: 672,
    isbn: '0-7475-8108-8'
  },
  {
    _id: ObjectId('68e373c710a9404ec288ceba'),
    title: 'Harry Potter and the Deathly Hallows',
    author: 'J.K. Rowling',
    year: 2007,
    genre: 'Fantasy Fiction',
    pages: 607,
    isbn: '0-7475-9105-9'
  },
  {
    _id: ObjectId('68e373c710a9404ec288cebb'),
    title: 'The Ballad of Songbirds and Snakes',
    author: 'Suzanne Collins',
    year: 2020,
    genre: 'Science Fiction',
    pages: 528,
    isbn: '9780702300172'
  },
  {
    _id: ObjectId('68e373c710a9404ec288cebc'),
    title: 'Sunrise on the Reaping',
    author: 'Suzanne Collins',
    year: 2025,
    genre: 'Science Fiction',
    pages: 400,
    isbn: '9781546171461'
  }
]
```

Atlas atlas-barxr0-shard-0 [primary] LibraryDB&gt;

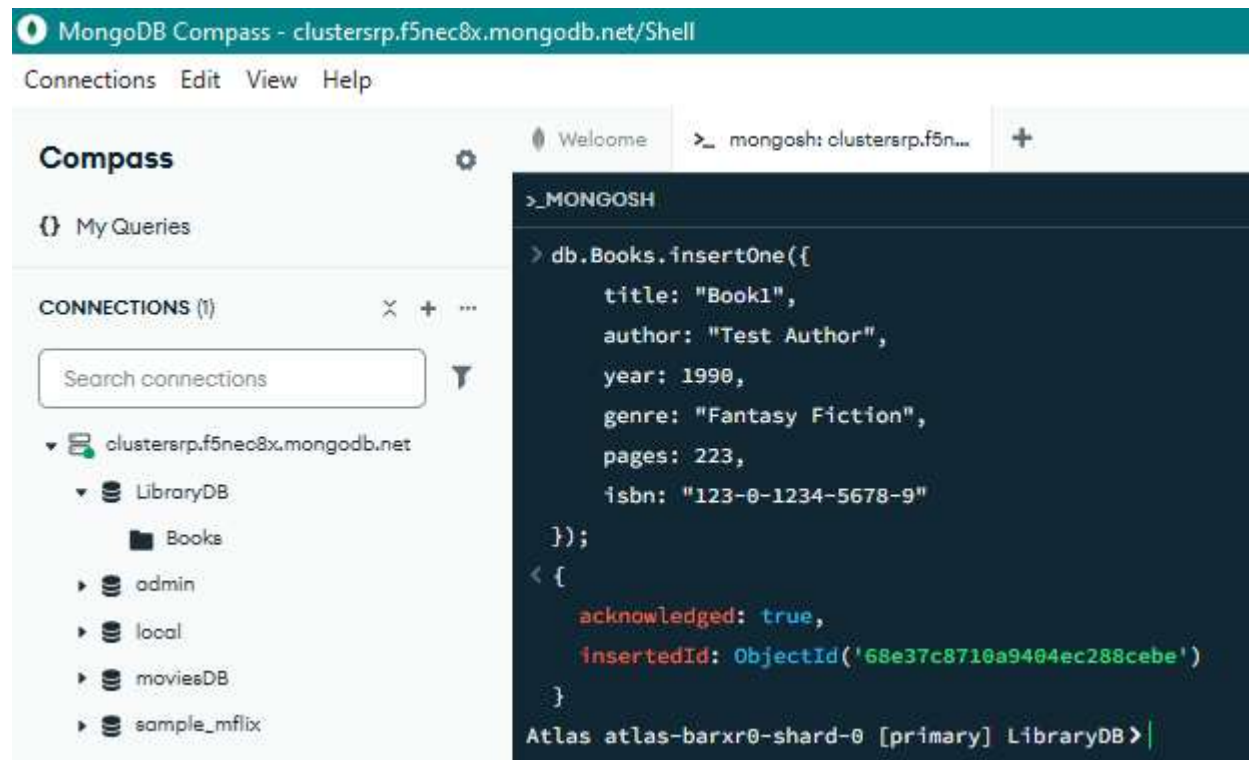
## Task 3. Updating and Deleting Documents in MongoDB

- a) Use the LibraryDB database and update the genre of the book titled "Book1" to "Classic Fiction".

Firstly, inserting the book named 'Book1' in the 'Books' collection:

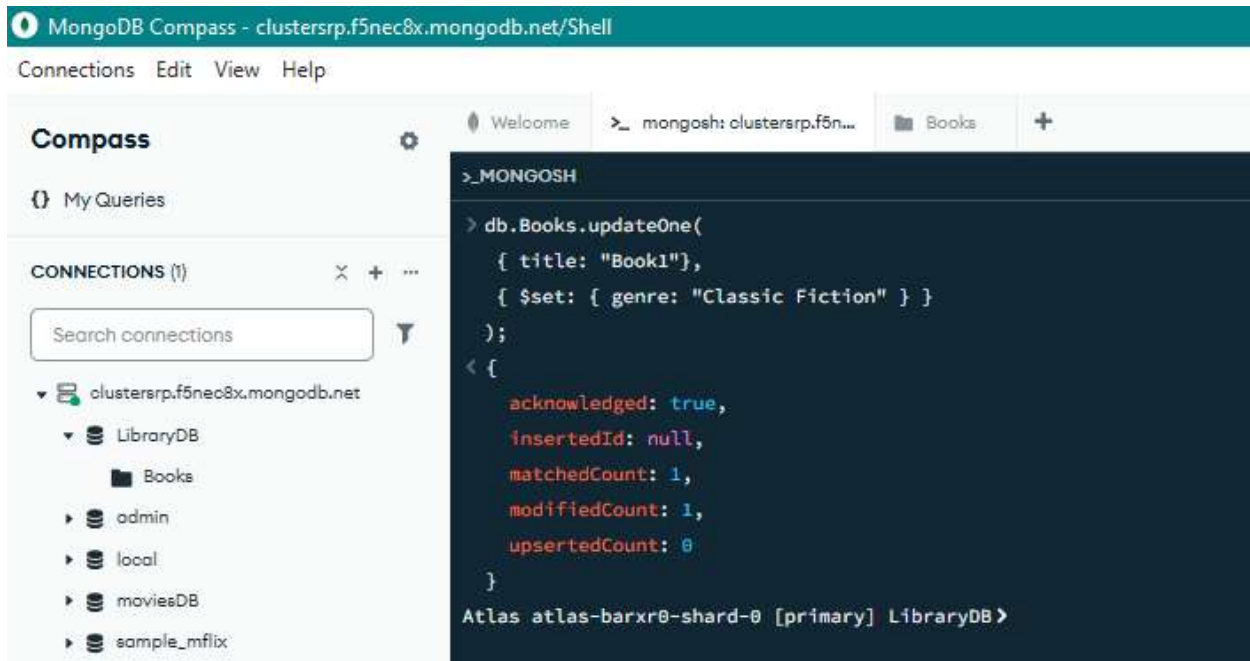
```
use LibraryDB;
```

```
db.Books.insertOne({  
  title: "Book1",  
  author: "Test Author",  
  year: 1990,  
  genre: "Fantasy Fiction",  
  pages: 223,  
  isbn: "123-0-1234-5678-9"  
});
```



Then updating the genre of the book titled "Book1":

```
db.Books.updateOne(  
  { title: "Book1"},  
  { $set: { genre: "Classic Fiction" } }  
);
```

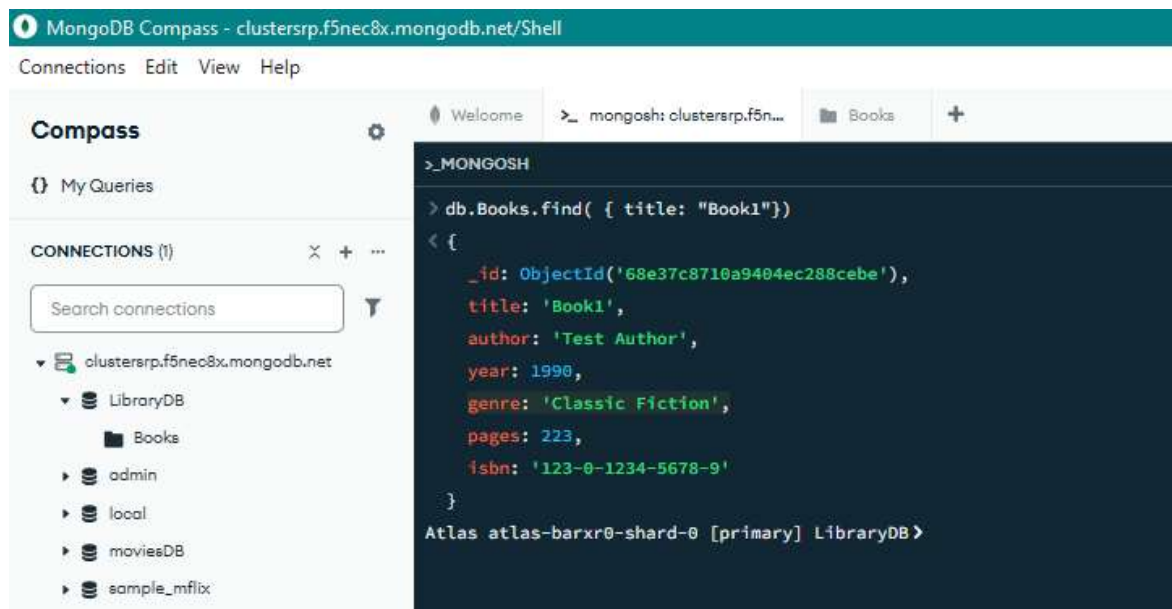


**a-c) Use find() to verify the updates.**

Below command is used to verify the genre of 'Book1' is updated correctly.

```
db.Books.find( { title: "Book1"})
```

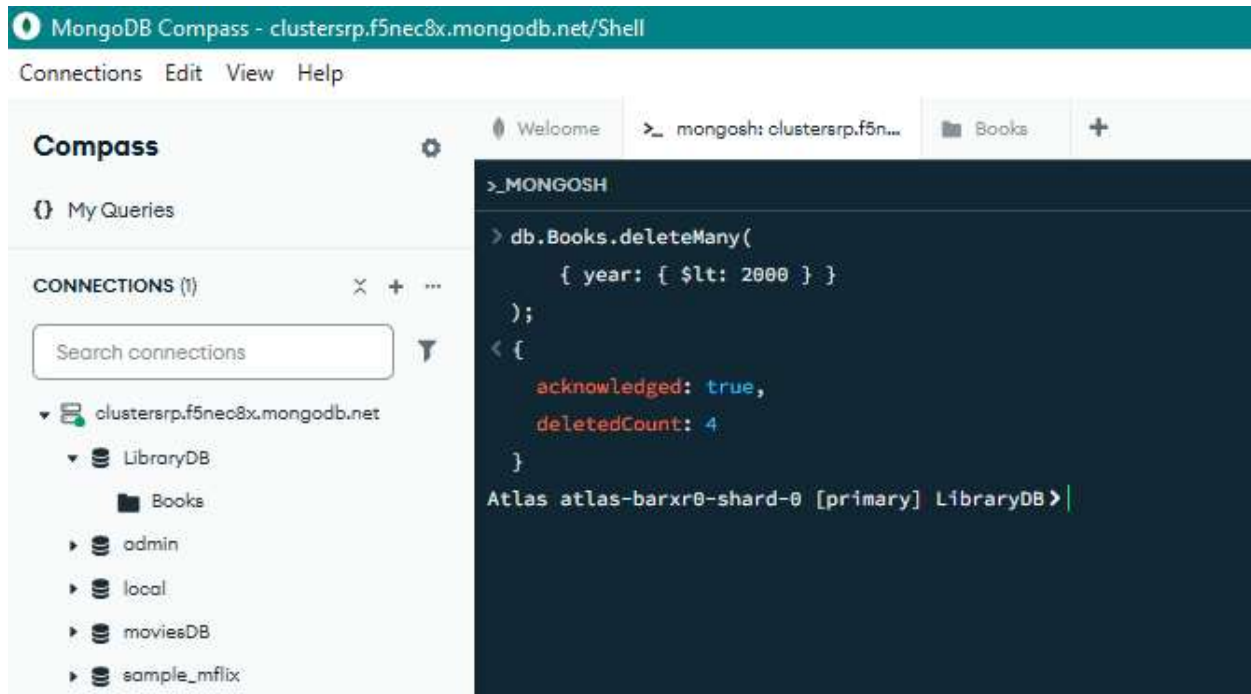
We can see that the genre of 'Book1' is successfully changed to 'Classic Fiction'.



**b) Delete any book in the collection where the year is less than 2000.**

Below command is used to delete any book in the collection where the year is less than 2000.

```
db.Books.deleteMany(  
  { year: { $lt: 2000 } }  
);
```



**b-c) Use find() to verify the deletions.**

Below command is used to verify the deletion of books with year less than 2000.

```
db.Books.find({ year: { $lt: 2000 } })
```

The below screenshot shows that there is no book with year less than 2000.

## Compass

My Queries

CONNECTIONS (1)

Search connections

clustersrp.f5nec8x.mongodb.net

LibraryDB

Books

admin

local

moviesDB

sample\_mflix

Welcome

mongosh: clustersrp.f5n...

Books

+

>\_MONGOSH

> db.Books.find({ year: { \$lt: 2000 } })

<

Atlas atlas-barxr0-shard-0 [primary] LibraryDB>